

18/08/26

CSA-1109 Object Oriented Analysis and Design

ASHMI.P.S

192571004

CO4 AT-1

Class Test

1) Explain how OOP principles support Layered Architecture.

Introduction:

Layered architecture software design approach in which a system divided into separate layers, where each layer has a specific responsibility. Common layers are Presentation Layer, Business Logic and Data Access Layer.

1. Encapsulation

⇒ Building data and the methods that operate on the data inside a class and restricting direct access to internal data.

* Data can be detected as private and accessed through methods.

* Each class controls its own data and behaviour.

* It prevents security and maintainability.

Example:

In a banking application, the bank account can keep balance private. Other classes cannot directly modify the balance, they must use methods such as `deposit()` and `withdraw()`.

Thus, encapsulation helps maintain clear boundaries between layers.

Abstraction

→ Hiding internal implementation details and showing only necessary functionality.

- * Reduces system complexity.
- * Allows one layer to use another layer without knowing how the database retrieves the data.
- * Makes changes easier.

Example:

The presentation layer calls `getCustomerDetails()` without knowing how the database retrieves the data.

Modularity:

→ Modularity means dividing a large system into small independent modules.

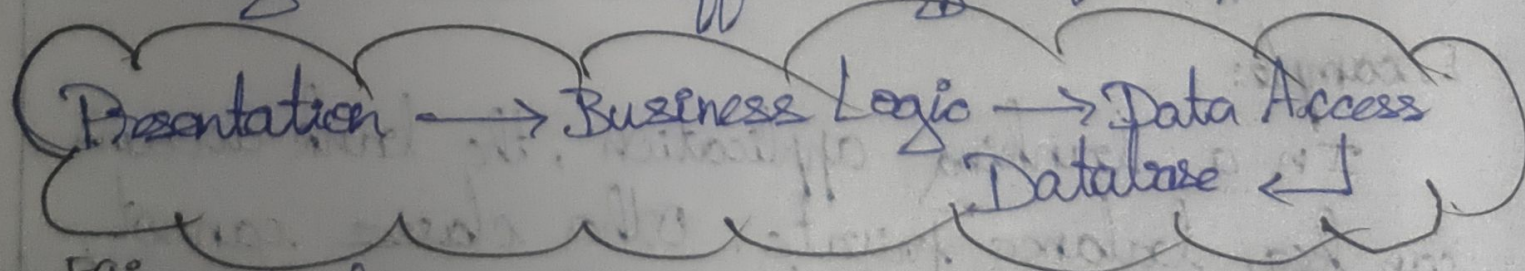
- * Each module has a specific responsibility.
- * Modules can be developed and tested separately.
- * Improves reusability and maintainability.

Eg:

E-commerce application can contain separate modules for: user management, order management and payment management, etc...

Separation of Concerns:

The main advantage is separation of concerns. Each layer handles a different type of responsibility.



Eg:

If changes remain. Changing the user interface from a web application to a mobile application should not require changing the business rules.

Maintainability:

- * UI ~~dev~~ changes remain mostly within the presentation layer.
- * Database changes remain within the data access layer.
- * Errors can be identified and fixed in the responsible layer.

Conclusion:

Therefore, encapsulation, abstraction and modularity provide the foundation for a well-structured layered architecture. They reduce coupling, improve separation of concerns, increase maintainability and make the software easier to test, modify and scale.

2) Roles and responsibilities of layers in Layered Architecture.

Introduction:

Layered architecture divides an application into multiple layers. The three important layers are: 1. Presentation Layer 2. Business Logic Layer 3. Data Access Layer. Each layer has a specific responsibility and communicates with the adjacent layer.

1. Presentation Layer

Responsible for interaction between the user and the application.

Responsibilities:

- * Provides the user interface
- * Accepts user input
- * Displays output and messages.

Example:

Web pages, buttons and menus

2. Business Logic Layer

The main rules and processing of the application.

Responsibilities:

- * Processes user requests.
- * Implement business rules.
- * Validates business conditions.

Eg: In an online shopping system, the business can check if the product is available before placing the order? and it can also calculate discounts, taxes and total price.

3. Data Access Layer

The data access layer is responsible for communication with the database or other data sources.

Responsibilities:

- * Connects to the data
- * Executes SQL queries.
- * Inserts, updates, deletes and retrieves data.

Example:

A customer DAO class may contain methods such as: add Customer(), get Customer().

Interaction: between layers:

User $\leftrightarrow$ Presentation Layer $\leftrightarrow$ Business Logic Layer $\leftrightarrow$ Data Access Layer $\leftrightarrow$ Database.

Real world example: Online Shopping System
Suppose a customer wants to purchase a product.
Presentation Layer: The customer clicks the Buy Now Button

Business Logic Layer: The system checks product availability, calculates price, discount, tax and validates the order.

Data Access Layer: The system retrieves product details and update the stock in the database.

Response: The result is returned through the business layer to the presentation layer.

Presentation: The user sees an order confirmation message.

Application:

- * Clear separation of responsibilities
- * Easier testing
- * Easier maintenance
- * Better security
- * Improved scalability.

Conclusion:

A layered architecture ensures that ensures each layer performs a well-defined task. The presentation layer handles user interaction, the business layer handles application rules and the data access layer handles data storage and retrieval.

3) UI directly accessing database and containing business logic.

Introduction:

Consider an application where the user interface directly communicates with database and also contains business logic.

For example:

User Interface $\rightarrow$ Business logic + Database

Problems in the Existing System.

Maintainability Issues:

When UI, business logic and database operations are mixed together.

- * Code becomes difficult to understand
- * UI code becomes very large and complex.
- * Change become difficult.
- * Debugging becomes harder.
- * Business rules may be repeated in multiple screens.

Eg:

If a discount rule changes from 10% to 15%, the developer may need to find and modify the discount calculation in multiple UI screens.

Scalability Issues

The system becomes difficult to scale because:

- * Difficult to add new features.
- * Difficult to support web, mobile and other interfaces.
- * Database operations may become inefficient.
- * The system becomes difficult to expand.

Eg: If a co
app and web

Security
Discrete dat
tasks.

* Direct
informa

* Database

* Unauth

* SQL

Main Pr

The

Therefore
changes

Proposed

Use

Pre

B

D

I

Present

Business

Data

Database

Eg: If a company wants to create both a mobile app and web application

Security Issues:

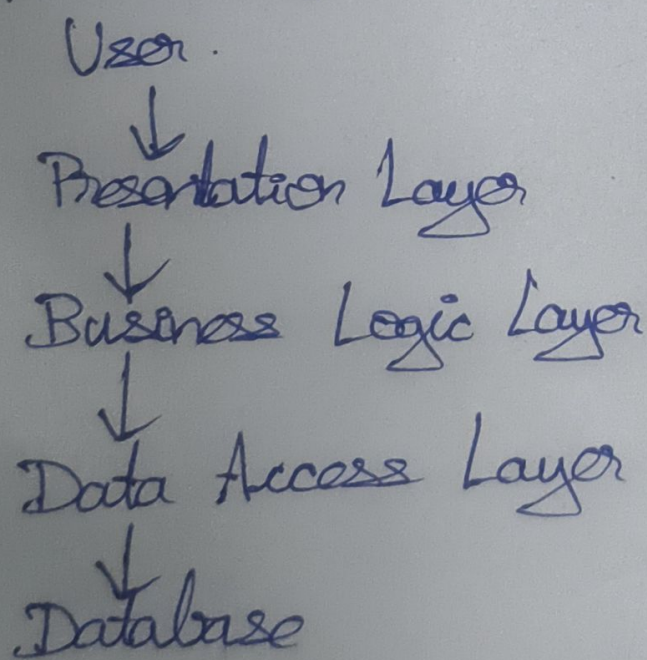
Direct database access from the UI creates serious risks.

- * Direct database access can expose sensitive information
- * Database credentials may be exposed.
- * Unauthorized queries may be executed.
- * SQL injection risks may increase.

Main Problem: Tight Coupling

The UI becomes strongly dependent on the database. Therefore, changing the database structure may require changes in the UI.

Proposed Layer Solution:



Presentation Layer → Handles UI and user input.

Business Logic Layer → Handles rules and processing

Data Access Layer → Handles database operations

Database → stores the application data.

Example:

In an online banking system:

Without layering:

Login Screen $\rightarrow$ SQL Database

The login screen directly executes SQL queries and contains authentication logic.

With layering:

Login Screen $\rightarrow$ Authentication Service $\rightarrow$ User DAO $\rightarrow$ Database.

The presentation layer sends login details to the business layer. The business layer applies authentication rules and the data access layer retrieves user information from the database.

Conclusion:

Layered architecture prevents direct dependency between UI and database, and makes the overall system easier to develop and maintain.

